

Aula prática 05 - Mapeamento de Texturas

Introdução à Computação Gráfica
Prof. Davi Santos

- Download dos códigos
- Documentação dos OpenGL 2.1
- Livro de referência
- LINUX: compile e execute os códigos com *g++ hello.c -o hello -lGL -lGLU -lglut && ./hello*

1 Geração de texturas - checker.c

O uso do mapeamento de textura no OpenGL 2.1 segue as seguintes etapas:

- Crie um objeto de textura e especifique uma textura para esse objeto;
 - **makeCheckImage()** no checker.c ou carrega a textura de um arquivo.
- Indique como a textura deve ser aplicada a cada pixel (ver **glTexParameter()** no kronos);
- Ative o mapeamento de textura;
 - **glEnable(GL_TEXTURE_2D)**
- Desenhe a cena, fornecendo a textura e as coordenadas geométricas.
 - A textura aplicada ao objeto é a que foi definida em **glBindTexture()** ou **glTexImage2D()**.

Para este primeiro exercício, siga os passos abaixo:

- Execute o código checker.c e observe a imagem renderizada;
- Remova o segundo quadrado, deixe apenas o da esquerda. Modifique os vértices do quadrado para que fique como a imagem 1;
- Aumente o número de quadrados na textura modificando os hexadecimais da linha 64 de 0x8 para 0x1;
- Troque os parâmetros GL_NEAREST por GL_LINEAR. o que aconteceu?
- Volte o hexadecimal para 0x8;
- Aumente a escala da imagem de textura mudando seus limites de (0.0, 0.0) ↔ (1.0, 1.0) para (0.0, 0.0) ↔ (10.0, 10.0);
- Mude o parâmetro GL_REPEAT para GL_CLAMP.

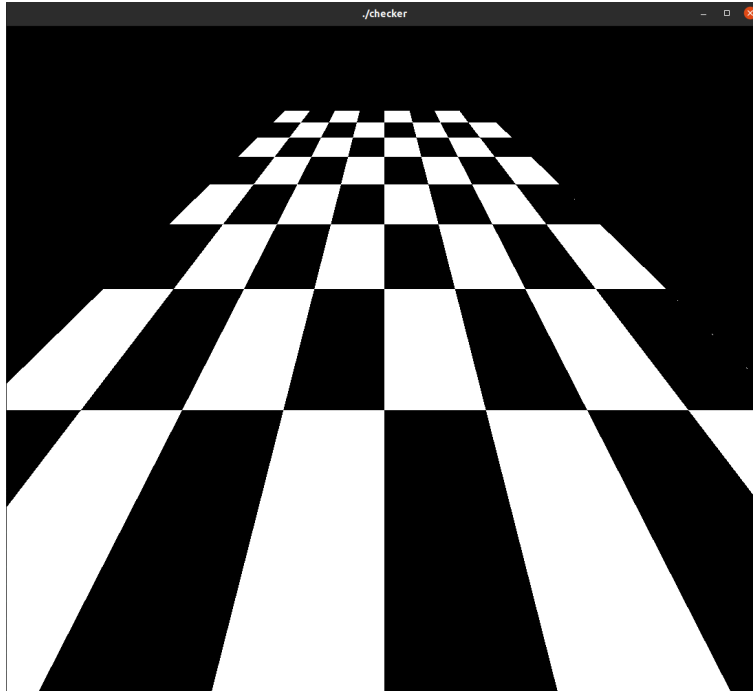


Figure 1: Quadrado esperado.

2 Carregando imagens de textura - checker.c

Para carregar imagens e usá-las como textura no OpenGL 2.1 é preciso baixar uma biblioteca externa.

- Baixe o código `stb_image.h` e coloque-o no mesmo diretório do `checker.c`;
- Inclua a biblioteca `stb_image` no começo do `checker.c` com `#include "stb_image.h"`;
- adicione a linha `#define STB_IMAGE_IMPLEMENTATION` antes de `#include "stb_image.h"`
- Adicione o método `loadTexture()` (Listing 1) ao código;
- Carregue a imagem no começo da função `display()` com `loadTexture("texture_image.jpg")`, onde `texture_image` é uma imagem (a sua escolha) que está no mesmo diretório de `checker.c` (pode ser no formato png, jpg, jpeg, etc);
- Antes de desenhar, pinte o objeto de vermelho;
- Troque `GL_DECAL` por `GL_REPLACE`, `GL_MODULATE`, `GL_BLEND` e `GL_ADD` e veja os resultados;
- Troque o objeto por uma esfera com os comandos abaixo e refaça os exercícios:

```
GLUquadric* quad = gluNewQuadric();
gluQuadricTexture(quad, GL_TRUE);
gluSphere(quad, 1.0, 32, 32);
```

Listing 1: Carregando uma imagem como textura.

```
void loadTexture(const char* filename) {
    int width, height, nrChannels;
    unsigned char* data = stbi_load(filename, &width, &height,
    &nrChannels, 0);
    if (data) {
        glGenTextures(1, &texture);
        glBindTexture(GL_TEXTURE_2D, texture);

        // Set texture wrapping and filtering parameters
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
        GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
        GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
        GL_LINEAR_MIPMAP_LINEAR);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
        GL_LINEAR);

        // Load the texture data (check if it's RGB or RGBA)
        if (nrChannels == 3) {
            gluBuild2DMipmaps(GL_TEXTURE_2D, GL_RGB, width,
            height, GL_RGB, GL_UNSIGNED_BYTE, data);
        } else if (nrChannels == 4) {
            gluBuild2DMipmaps(GL_TEXTURE_2D, GL_RGBA, width,
            height, GL_RGBA, GL_UNSIGNED_BYTE, data);
        }

        stbi_image_free(data);
    } else {
        std::cerr << "Failed to load texture: " << filename <<
        std::endl;
    }
}
```